# Прикладная теория типов

Домашнее задание 3 (полиморфизм и зависимые типы)

Осень 2026

---

Домашняя работа принимается до 23:59 1 декабря 2026, кроме задач, помеченных звёздочкой, которые принимаются до конца семестра. Решения можно набрать в TEX или написать разборчивым текстом на бумаге и отсканировать. Домашняя работа принимается в виде **одного** PDF-файла (кроме задач, где нужно сдавать код или в тексте условия есть упоминание про отдельный PDF-файл) на почту m.voronov@gse.cs.msu.ru. Вопросы по домашнему заданию можно задавать по почте или в ТГ-группе курса.

🏆 Задачи с кубком участвуют в топе задач. Бонус от максимального балла за соответствующую задачу: 1-е место — +100%, 2-е — +50%, 3-е — +25%. На каждом месте могут одновременно быть несколько студентов. Места определяются отдельно для каждой задачи, критерий субъективный в зависимости от ясности, оригинальности и изящества решения.

1. (1 балл) Сколько существует $\lambda 2$ контекстов для следующего списка деклараций: $\alpha : \star,\ \beta : \star,\ \gamma : \star,\ f : \alpha \to \beta,\ g : \gamma \to \beta,\ x : \beta$. Приведите обоснование.

2. Для каждого предтерма определите, типизуется ли он $\lambda 2$, если да - найдите его тип а-ля Карри и а-ля Чёрч, если нет - обоснуйте, почему предтерм не типизируем:

   - (2 балла) $\lambda x.(xx)(xx)$
   - (2 балла) $S \equiv \lambda xyz.xz(yz)$

3. Используя соответствие Карри-Ховарда и OCaml, докажите следующие логические утверждения пропозициональной логики. При реализации используйте произведение (`'a * 'b`) для $\wedge$ и сумму (например, `type ('a,'b) either = Left of 'a | Right of 'b`) для $\vee$.

   - (2 балла) $A \vee B \Longrightarrow B \vee A$
   - (2 балла) $(A \wedge B) \Longrightarrow (D \vee C) \Longrightarrow (D \Longrightarrow F) \Longrightarrow (C \Longrightarrow G) \Longrightarrow ((A \wedge F) \vee (B \wedge G))$
   - (2 балла) $((C \Longrightarrow A) \wedge (C \Longrightarrow B)) \Longrightarrow (C \Longrightarrow (A \wedge B))$

4. Используя соответствие Карри-Ховарда и OCaml, покажите следующие изоморфизмы типов (под $a \sim b$ понимается наличие прямой и обратной функций, которые являются взаимными обратными). Для каждого изоморфизма: (a) переведите в типы OCaml, (b) реализуйте обе функции с явными типами, (c) объясните, почему они взаимно обратны.

   - (3 балла) $a^{b+c} \sim a^b \times a^c$
   - (3 балла) $(a^b)^c \sim a^{b \times c}$
   - (2 балла) $a^0 \sim 1$
   - (2 балла) $a^1 \sim a$

5. Рассмотрим конструктор типов:

$$\mathsf{Flip} \equiv \lambda F : \star \to \star \to \star.\, \lambda \alpha : \star.\, \lambda \beta : \star.\, F\, \beta\, \alpha$$

   - (1 балл) Определите вид конструктора Flip
   - (1 балл) Постройте полное дерево вывода вида в системе $\lambda\omega$ с использованием правил из лекции 9
   - (1 балл) Приведите пример применения: чему равен Flip Either Int Bool? Укажите вид результата.

6. Рассмотрим следующие определения в System $F_\omega$:

   - $\mathsf{Functor}\ F \equiv \forall \alpha\, \beta.\, (\alpha \to \beta) \to F\,\alpha \to F\,\beta$
   - *fmapList* : $\mathsf{Functor\ List}$ — реализация для списков
   - *fmapMaybe* : $\mathsf{Functor\ Maybe}$ — реализация для Maybe
   - (1 балл) Какой вид должен иметь параметр $F$ в определении $\mathsf{Functor}$?
   - (2 балла) Докажите, что если $F$ и $G$ — функторы, то $\mathsf{Compose}\,F\,G$ — тоже функтор. Приведите определение *fmapCompose* через $fmap_F$ и $fmap_G$.
   - (1 балл) Объясните, почему в Java/C# нельзя выразить такую абстракцию, а в Haskell можно. Какого механизмы не хватает языкам.

7. Используя систему $\lambda P$ и зависимые типы:

   - (1 балл) Спроектируйте и формализуйте тип функции `replicate`, которая создаёт вектор длины $n$ из $n$ копий элемента. Используйте П-типы и тип $\mathsf{Vec}\,A\,n$ (вектор элементов типа $A$ длины $n$).
   - (1 балл) Спроектируйте и формализуйте тип функции `safeHead`, которая извлекает первый элемент непустого вектора. Как типы гарантируют, что вектор точно непустой?
   - (1 балл) Объясните, почему попытка вызвать `safeHead` на пустом векторе `nil` приведёт к ошибке типизации.
   - (1 балл) Приведите пример корректного вызова `safeHead` с явным указанием всех аргументов и их типов.

8. Рассмотрим Σ-типы в системе $\lambda P$:

   - (1 балл) Объясните разницу между типами $A \times \mathsf{Proof}$ и $\Sigma x : A.P(x)$ на конкретном примере.
   - (1 балл) Спроектируйте тип функции `filter`, которая фильтрует список по предикату и возвращает новый список. Проблема: длина результата заранее неизвестна. Как использовать Σ-тип, чтобы возвращаемое значение включало и сам результат, и его длину?
   - (1 балл) Формализуйте утверждение «существует чётное число больше 2» как Σ-тип.
   - (1 балл) Постройте обитатель этого типа (т.е. докажите утверждение).

9. Объясните простыми словами, понятными семилетнему ребёнку:

   - (3 балла) Что такое виды (kinds) и зачем они нужны. Почему мы не можем написать $\mathsf{List\ Maybe}$, но можем написать $\mathsf{List\ Int}$ и $\mathsf{Compose\ List\ Maybe}$? Приведите бытовую аналогию (например, с коробками, функциями, рецептами и т.п.).
   - (3 балла) Что такое зависимые типы на примере $\mathsf{Vec}\,A\,n$ (вектор длины $n$). Чем $\mathsf{Vec\ Int}\,3$ отличается от обычного $\mathsf{List\ Int}$? Почему компилятор не даст нам вызвать `head` на пустом векторе, но позволит на непустом? Приведите аналогию из повседневной жизни.

10. 🏆 **Типобезопасный ML-pipeline: от базовых гарантий до защиты от adversarial attacks.**

Требуется спроектировать систему типов для безопасной библиотеки машинного обучения, которая гарантирует корректность и устойчивость к атакам на уровне типов:

- **Базовые гарантии:** предотвращение shape mismatch, использования необученных моделей, data leakage.
- **Безопасность данных:** разделение adversarial и clean данных, защита от data poisoning.
- **Privacy:** отслеживание применения differential privacy и sensitive data.

*Контекст:* В production ML-системах критически важно предотвратить не только технические ошибки, но и атаки: adversarial examples, data poisoning, model inversion, membership inference. Типы — один из вариантов этой защиты.

**Язык:** код для решения этой задачи рекомендуется писать на любом языке, который поддерживает ADT/GADT, помимо этого Вы можете использовать Ваш любимый язык в предположении, что в нём есть ADT/GADT, т.е. по сути псевдокод, основанный на каком-то языке, а также можете напрямую использовать любой понятный псевдокод.

*Замечание о структуре:* Подпункты 1–6 представляют собой независимые аспекты безопасности ML-систем. Каждый можно решать отдельно, используя одну технику (phantom types и/или GADT). Подпункт 7 требует систематического анализа всех реализованных инвариантов. Опционально: попробуйте объединить несколько модулей (например, dataset с privacy и lifecycle модели) в единую систему типов — это усилит Ваше решение и может дать дополнительные баллы.

*Оценивание задания:* Это задание участвует в субъективном конкурсе, как и задание с объяснением теорем из ДЗ1. Будет оцениваться полнота реализации, объяснений и креатива: за первое место можно получить +50% баллов, за второе — +30%, за третье — +20% от общего числа баллов.

**Формат сдачи:** Сдайте ссылку на публичный репозиторий (GitHub/GitLab) с кодом и README, либо архив с исходниками. Обязательно приложите отдельный PDF-файл с объяснениями дизайна системы типов, примерами использования и анализом инвариантов. Альтернативно: можно включить все объяснения в виде подробных комментариев прямо в коде, если они достаточно развёрнутые.

- (4 балла) **Shape-safe архитектура.** Спроектируйте систему типов для слоёв нейронной сети, которая гарантирует согласованность размерностей на этапе компиляции.
  **Требования:**
  - Тензоры должны быть параметризованы размерностями (например, batch size и число features)
  - Слои нейросети (Dense, ReLU, Dropout) должны иметь типы, отражающие их входную и выходную размерность
  - Композиция слоёв должна быть типобезопасной: компилятор должен отвергать попытку соединить слой с выходом 128 и слой с входом 256
  - Используйте phantom types и/или GADT

  **Что нужно сделать:**
  - Определите типы для тензоров и слоёв
  - Опишите типы для 2-3 конкретных слоёв (Dense, ReLU, и др.)
  - Определите функцию композиции слоёв с корректными типами
  - Приведите пример некорректной архитектуры, которая будет отвергнута компилятором
  - Объясните, как phantom types и/или GADT обеспечивают проверку согласованности размерностей
- (3 балла) **Lifecycle модели и состояния.** Спроектируйте систему типов для отслеживания состояния модели машинного обучения.
  **Требования:**
  - Модель должна проходить через состояния: Untrained → Trained → Validated
  - Нельзя делать предсказания на необученной модели
  - Нельзя повторно обучить уже обученную модель (без явного сброса)
  - Валидация возможна только после обучения

– Используйте phantom types и/или GADT для кодирования состояний

**Что нужно сделать:**

– Определите типы для модели с состояниями (например, `('inp, 'out, 'state) model`)

– Спроектируйте типы функций init, train, validate, predict так, чтобы они гарантировали корректный lifecycle

– Приведите 2-3 примера некорректных вызовов, которые будут отвергнуты компилятором

– Объясните, какие инварианты безопасности закодированы в типах

- (3 балла) **Типобезопасные датасеты и защита от data leakage.** Спроектируйте систему типов для отслеживания разбиения данных и их обработки.

  **Требования:**

  – Датасеты должны различаться по split'у: Train, Validation, Test

  – Данные могут быть Raw (сырые) или Normalized (обработанные)

  – Нельзя обучить модель на Test-данных

  – Нельзя подать необработанные (Raw) данные в функцию, требующую Normalized

  – Нормализация параметров должна вычисляться только на Train-данных

  **Что нужно сделать:**

  – Спроектируйте параметризованный тип для датасетов (используя phantom types)

  – Определите типы функций: загрузка данных, нормализация, разбиение на train/test, обучение модели

  – Покажите, как типы гарантируют, что:

    * модель обучается только на Train-данных
    * в обучение подаются только Normalized данные
    * Test-данные не попадают в обучение

  – Приведите примеры некорректного кода, который отвергнет компилятор

- (2 балла) **Связь с $\lambda P$ и зависимыми типами.** Выберите один из инвариантов из предыдущих подпунктов (например, «нельзя вызывать predict на необученной модели» или «модель обучается только на Train-данных») и:

  – Запишите его как формулу в стиле $\lambda P$ (используя $\Pi$- и, при необходимости, $\Sigma$-типы).

  – Объясните, как ваш phantom-type дизайн в ML-языке приближённо реализует этот инвариант, и почему без полноценных зависимых типов (как в $\lambda P$) он не может быть выражен ровно.

- (3 балла) **Защита от adversarial examples.** Adversarial examples — это специально сгенерированные входы, которые обманывают модель. В production системах критично не смешивать adversarial данные с чистыми при обучении.

  **Требования:**

  – Данные должны быть помечены как Clean, Adversarial или Suspicious

  – Обычная функция train не должна принимать Adversarial данные

  – Adversarial training должен быть отдельной операцией, принимающей и clean и adversarial примеры

  – Adversarial примеры должны генерироваться только из clean данных с помощью модели

  – Функция evaluation должна работать с данными любого происхождения

  **Что нужно сделать:**

  – Спроектируйте параметризованный тип для данных с меткой происхождения

  – Определите типы функций: загрузка clean данных, генерация adversarial примеров, adversarial training, evaluation

  – Объясните, как типы предотвращают случайное смешивание adversarial и clean данных при обычном обучении

  – Приведите пример реальной атаки (FGSM, PGD, DeepFool) и объясните, как ваш дизайн типов помогает безопасно работать с такими данными

– (Бонус) Как расширить систему для защиты от data poisoning (вредоносных меток)?

- (4 балла) **Privacy-aware типы.** Differential privacy — методика добавления шума в данные/модель для защиты приватности пользователей.
  **Требования:**
  – Данные должны быть помечены как Public или Private с параметром приватности $\epsilon$
  – Нельзя опубликовать (release) приватные данные без достаточного уровня защиты
  – Система должна отслеживать privacy budget при композиции операций
  – Тип должен отражать уровень приватности
  **Что нужно сделать:**
  – Спроектируйте параметризованный тип для данных с уровнем приватности
  – Определите типы функций для добавления шума и публикации данных
  – Объясните, как типы могут отслеживать privacy budget (композицию операций с разными $\epsilon$)
  – Обсудите, почему это близко к зависимым типам (тип зависит от значения $\epsilon$)
  – Приведите пример из реальности: Google RAPPOR, Apple differential privacy, или federated learning
- (2 балла) **Систематический анализ инвариантов безопасности.** На основе вашей реализации из пунктов 1-6:
  – Составьте полный список инвариантов безопасности, которые гарантируются вашими типами (минимум 5-6 инвариантов). Для каждого укажите: что запрещается, какой тип это обеспечивает, какую ошибку даст компилятор.
  – Обсудите ограничения: какие важные инварианты НЕ выразимы в вашей системе типов и требуют runtime проверок, тестов или зависимых типов? Например:
    * "Accuracy модели не меньше baseline"
    * "Dataset сбалансирован (одинаковое число примеров каждого класса)"
    * "Нет дубликатов между train и test"

11.* (6 баллов) Для каждого терма исследуйте типизируемость в System F в стиле Карри и приведите формальное обоснование. Вычислите $\beta$-нормальную форму и сопоставьте результат вычисления с результатом анализа типов. Здесь $I \equiv \lambda x.x$ и $K \equiv \lambda xy.x$.

- $(\lambda sz.s(sz))(\lambda sz.s(sz))K$.
- $(\lambda zy.y(zI)(zK))(\lambda x.xx)$.

12.* **Параметричность.** Во всех пунктах рассматриваются замкнутые термы чистой System F с обычной кодировкой конечных списков. Сформулируйте используемое свойство параметричности.

- (2 балла) Опишите все термы типа $\forall\alpha.\alpha \to \alpha$ с точностью до $\beta\eta$-эквивалентности. Обоснуйте полноту описания.
- (4 балла) Пусть $m : \forall\alpha\,\beta.(\alpha \to \beta) \to \mathsf{List}\,\alpha \to \mathsf{List}\,\beta$. Исследуйте, следует ли из этого типа равенство

$$m[A, C]\,(g \circ f) = m[B, C]\,g \circ m[A, B]\,f$$

  для любых $f : A \to B$ и $g : B \to C$. Приведите доказательство или замкнутый контрпример с конкретными аргументами. Затем исследуйте то же равенство при дополнительном условии

$$m[A, A]\,id_A = id_{\mathsf{List}\,A} \qquad \text{для каждого типа } A.$$

  Укажите, где в обосновании используется параметричность. Равенства операций над списками понимаются экстенсионально.
- (2 балла) Опишите все термы типа $\forall\alpha.(\alpha \to \alpha) \to \alpha \to \alpha$ с точностью до $\beta\eta$-эквивалентности. Обоснуйте полноту описания и объясните вычислительный смысл этих термов.

14.* **Язык выражений и GADT.** Дан синтаксис языка:

```
type expr =
  | Int of int
  | Bool of bool
  | String of string
  | Add of expr * expr
  | Concat of expr * expr
  | Eq of expr * expr
  | If of expr * expr * expr
  | Length of expr
```

Операция `Add` складывает целые числа, `Concat` соединяет строки, `Length` вычисляет длину строки. Операция `Eq` сравнивает значения одного базового типа. В условном выражении условие имеет булев тип, а обе ветви – один общий тип.

- (2 балла) Предложите тип результата интерпретатора для данного ADT и способ сообщать об ошибках выражения. Приведите примеры вычисления и перечислите проверки, которые выполняет интерпретатор.
- (3 балла) Спроектируйте GADT `'a typed_expr`, отражающий тип результата выражения. Сохраните имена конструкторов и запись нескольких аргументов через кортежи. Для каждого выражения ниже исследуйте возможность его представления в вашем GADT. Ответы обоснуйте типизацией или диагностикой компилятора.

  ```
  Add (Int 5, String "hello")
  Concat (String "a", String "b")
  If (Eq (Int 5, Int 10), String "yes", String "no")
  If (Int 42, Int 1, Int 2)
  Add (Int 5, If (Bool true, Int 1, Int 2))
  Length (If (Eq (Int 1, Int 1), String "a", String "b"))
  ```

- (2 балла) Сформулируйте тип интерпретатора для вашего GADT и реализуйте его на OCaml. Объясните роль индекса типа при разборе конструкторов. Сопоставьте эту реализацию с интерпретатором исходного ADT.
- (1 балл) Найдите применение GADT в библиотеке или компиляторе. Приведите ссылку на конкретный тип и разберите свойство, которое выражает его индекс. Сопоставьте этот способ описания свойства с зависимыми типами.

15.* **Формальный анализ нарушения soundness в системе типов компилятора Rust**

Требуется провести исследование дефекта в системе типов компилятора Rust, приводящего к нарушению свойства soundness (корректности). Под нарушением soundness понимается ситуация, когда система типов допускает компиляцию программы, нарушающей инварианты безопасности памяти или типов, что противоречит теоретическим гарантиям типизированных языков программирования.

**Цель работы:** исследовать взаимосвязь между теоретическими основами систем типов (полиморфизм, изоморфизм Карри-Ховарда, soundness, coherence, леммы progress и preservation) и их практической реализацией в промышленных компиляторах, а также проанализировать последствия нарушения теоретических свойств для безопасности программного обеспечения.

**Критерий оценки:** за это задание также можно получить до +200% дополнительных баллов, если анализ будет субъективно исчерпывающим, Вы реально выберете сложный недостаток, напишите подробную статью в свой блог/habr/medium/... и за любой креатив помимо того, что требуется в самом задании. Критерий выставления дополнительных баллов - субъективный, но Вы можете связаться с лектором и уточнить, как и что может быть оценено.

**Критерии выбора дефекта**

Необходимо выбрать один дефект из официального репозитория компилятора Rust (rust-lang/rust), удовлетворяющий следующим требованиям:

- Дефект классифицирован метками T-types (относящийся к подсистеме типов) и C-bug (подтверждённый дефект)
- Дефект находится в открытом состоянии либо был закрыт не ранее чем за 12 месяцев до момента сдачи работы
- Дефект приводит к нарушению soundness системы типов, формально определяемому как возможность получения undefined behavior без использования конструкций языка, помеченных ключевым словом unsafe

**Методика поиска:** рекомендуется использовать систему отслеживания дефектов GitHub Issues с фильтром `is:issue state:open label:C-bug label:T-types`. Для уточнения поиска рекомендуется добавить метку `label:I-unsound`, явно маркирующую дефекты, нарушающие soundness. Допускается выбор недавно закрытых дефектов при недостаточном количестве открытых.

*Замечание:* При наличии неопределённости относительно соответствия выбранного дефекта требованиям задачи или его сложности допускается предварительное согласование выбора с преподавателем.

**Формат сдачи:** Сдайте PDF-документ с формальным анализом дефекта. Для дополнительной части (эксплуатация дефекта) приложите ссылку на Rust Playground, ссылку на репозиторий GitHub/GitLab с исходным кодом .rs файла, либо сам .rs файл. Весь код должен быть снабжён детальными комментариями, объясняющими каждый этап эксплуатации.

**Часть 1: Формальный анализ дефекта**

Анализ должен включать следующие обязательные компоненты:

- **Формальная спецификация дефекта:**
  - Идентификатор дефекта в системе GitHub Issues с полной ссылкой
  - Формализованное описание проблемы с использованием терминологии теории типов
  - Версия компилятора, в которой дефект воспроизводится (если установлено)
  - Минимальный воспроизводимый пример (minimal reproducible example, MRE) — корректно компилирующаяся программа, демонстрирующая дефект
- **Технический анализ причин дефекта:**
  - Идентификация компонента системы типов, содержащего дефект (trait resolution, lifetime inference, variance checking, borrow checker, coherence, specialization, и т.д.)
  - Формальное объяснение причины некорректного поведения компилятора: отсутствующая либо некорректная проверка типизации
  - Установление связи с теоретическими концепциями: определение нарушенных свойств системы типов (progress, preservation, parametricity, coherence)

  - Анализ предложенных в обсуждении дефекта гипотез о его причинах и возможных методах устранения (при наличии)
- **Анализ практических последствий:**
  - Классификация нарушенных гарантий безопасности (memory safety, thread safety, type safety)
  - Перечисление возможных проявлений undefined behavior (use-after-free, data races, type confusion, dangling references, invalid values)
  - Оценка вероятности случайного проявления дефекта в производственном коде
  - Конструирование конкретного примера программы, в которой дефект приводит к undefined behavior или нарушению инвариантов безопасности
- **Теоретико-типовой анализ:**
  - Формализация дефекта в терминах System F, System $F_\omega$ или зависимых типов
  - При наличии связи с trait-системой: описание в терминах type classes либо экзистенциальных типов
  - Анализ соотношения дефекта с фундаментальными леммами progress и preservation для типизированных исчислений

**Важно:** Работы, представляющие собой пересказ содержания issue без самостоятельного анализа и формализации, будут оцениваться на минимальном уровне. Требуется демонстрация понимания теоретических основ и способности применять формальные методы анализа.

**Часть 2: Эксплуатация дефекта в изолированной среде**

Опционально можно продемонстрировать методы эксплуатации дефекта в контексте изолированной среды исполнения (sandbox), в которой программа может исполнять произвольный Rust-код (например, Rust Playground, система плагинов), но запрещено использование конструкций unsafe, Foreign Function Interface (FFI) и системных вызовов для доступа к файловой системе или сети.

- **Формальное описание метода эксплуатации:** детализированное объяснение методологии использования нарушения soundness для получения возможности исполнения произвольного кода (arbitrary code execution) или доступа к произвольным областям памяти. Должно включать формализацию этапов эксплуатации и теоретическое обоснование корректности метода.
- **Работающая реализация эксплуатации (Proof of Concept):** программа, корректно компилирующаяся стабильной версией компилятора Rust без использования unsafe, демонстрирующая одно из следующих проявлений undefined behavior:
  - Чтение или модификация произвольных областей памяти
  - Вызов произвольной функции посредством некорректного указателя vtable
  - Конструирование значения типа с нарушенными инвариантами (например, значение типа bool, отличное от 0 и 1)
  - Нарушение правил алиасинга (aliasing), приводящее к состоянию гонки (data race) в однопоточной программе

*Замечание:* Требование полноценной демонстрации удалённого исполнения кода (remote code execution) не предъявляется. Достаточно убедительной демонстрации проявления undefined behavior в ограниченном сценарии.

**Классификация типов дефектов soundness** (допускается выбор дефекта любой категории):

- Нарушения когерентности trait-системы (trait coherence violations): конфликтующие реализации (impl) для одного типа, нарушающие свойство единственности (uniqueness)
- Дефекты вывода времён жизни (lifetime soundness): некорректный алгоритм вывода времён жизни (lifetime inference), допускающий создание висящих ссылок (dangling references)
- Нарушения правил вариантности (variance issues): некорректное определение ковариантности/контравариантности обобщённых параметров типов
- Дефекты специализации (specialization bugs): ошибки в механизме перекрывающихся реализаций при активированной возможности `min_specialization`

- Дефекты ограничений высшего ранга (HRTB — Higher-Rank Trait Bounds): некорректная обработка ограничений вида `for<'a>`
- Дефекты обобщения по константам (const generics): нарушения soundness в типах, параметризованных константами
- Дефекты обобщённых ассоциированных типов (GATs — Generic Associated Types): ошибки в реализации недавно стабилизированного механизма
- Дефекты проверки корректности деструкторов (drop check): некорректные проверки безопасности исполнения деструкторов

**Требования к оформлению**

- **Часть 1:** PDF-документ, содержащий формальный анализ (допускается объединение с решениями других задач в единый документ). Рекомендуемый объём: 1-2 страницы для базового уровня, 2-3 страниц для глубокого анализа.
- **Часть 2 (при наличии PoC):** исходный код на языке Rust (файл с расширением .rs) либо постоянная ссылка на Rust Playground, сопровождённые детальными комментариями, формализующими каждый этап эксплуатации.

**Рекомендуемая структура документа:**

(a) Введение: формулировка проблемы и обоснование её значимости для теории и практики типизированных языков

(b) Спецификация дефекта: MRE с формальным описанием некорректного поведения компилятора

(c) Технический анализ: идентификация нарушенного компонента системы типов и причин некорректного поведения

(d) Теоретический анализ: формализация в терминах теории типов (soundness, progress, preservation, System F/$F_\omega$)

(e) Анализ практических последствий: классификация возможных проявлений undefined behavior

(f) Заключение: обсуждение возможных методов устранения дефекта, выводы

(g) Приложение (опционально): демонстрация эксплуатации с формальными комментариями

**Критерии оценивания**

**Часть 1 (максимум 30 баллов):**

- **16–20 баллов (удовлетворительный уровень):** предоставлен MRE с описанием дефекта, поверхностное объяснение проблемы, минимальное использование формального аппарата теории типов. Работа носит преимущественно описательный характер без глубокого анализа.
- **22–26 баллов (хороший уровень):** детальное техническое объяснение причин некорректного поведения компилятора, систематический анализ практических последствий, явное установление связей с фундаментальными концепциями (soundness, progress, preservation). Наличие самостоятельно разработанных примеров и иллюстраций.
- **28–30 баллов (отличный уровень):** всё вышеперечисленное плюс глубокий формальный анализ в терминах теории типов (System F, System $F_\omega$, type classes, экзистенциальные типы), самостоятельные наблюдения относительно природы дефекта, обсуждение возможных методов устранения с формальным обоснованием. Демонстрация понимания нарушенных инвариантов на формальном уровне.

**Часть 2 (максимум 20 баллов):**

- 8 баллов: формальное описание метода эксплуатации с детализацией этапов
- 12 баллов: работающая реализация эксплуатации, демонстрирующая undefined behavior без использования unsafe
- Максимальная оценка: 50 баллов (30 за часть 1 + 20 за часть 2)

**Рекомендуемая литература**

- The Rustonomicon. Официальная документация — формальное описание семантики unsafe Rust и инвариантов безопасности
- The Rust Reference. «Subtyping and Variance» — формальная спецификация правил вариантности в системе типов Rust
- GitHub Issues: метка I-unsound — реестр известных дефектов, нарушающих soundness
- Counterexamples in Type Systems. counterexamples.org — сборник soundness bugs в различных языках программирования с объяснениями и примерами
- Jung R. et al. «RustBelt: Securing the Foundations of the Rust Programming Language», POPL 2018 — формальная верификация безопасности Rust в системе Coq
- Matsakis N. «Baby Steps» — технический блог о теоретических основах дизайна системы типов Rust

16.* 🏆 **Взаимная рекурсия, алгебры типов и катаморфизмы.**

Даны два полиномиальных функтора от двух аргументов:

$$F(X,Y) = \sum_{i \in I} A_i \times X^{p_i} \times Y^{q_i}, \qquad G(X,Y) = \sum_{j \in J} B_j \times X^{r_j} \times Y^{s_j}.$$

Множества $I, J$ конечны, показатели – натуральные числа, включая ноль. Типы меток $A_i, B_j$ произвольны. Степени здесь обозначают конечные произведения. Действие $F$ и $G$ на паре функций применяется к соответствующим полям конструкторов.

Рассмотрим взаимно рекурсивные типы $T, U$, состоящие из конечных значений, с конструкторами

$$\mathsf{in}_T : F(T,U) \to T, \qquad \mathsf{in}_U : G(T,U) \to U.$$

Это совместное наименьшее решение уравнений $T \cong F(T,U)$ и $U \cong G(T,U)$. Для одноаргументного функтора $H$ запись $\mu Z.H(Z)$ обозначает носитель начальной алгебры. $\mathsf{cata}_H$ – её катаморфизм, как в лекции 4. Работайте с чистыми тотальными функциями и экстенсиональным равенством.

- (4 балла) **Устранение взаимной рекурсии.** Выведите представления $\widehat{T}$ и $\widehat{U}$ через вложенные применения $\mu$, каждое из которых связывает одну типовую переменную. Для каждого введённого конструктора типов задайте действие на функциях и докажите законы тождества и композиции. Конструкция должна работать для любых $F, G$ указанного вида.
- (4 балла) **Изоморфизмы.** Постройте преобразования $T \rightleftarrows \widehat{T}$ и $U \rightleftarrows \widehat{U}$. Выпишите их типы и уравнения на конструкторах, докажите взаимную обратимость. Обоснуйте завершение преобразований.
- (6 баллов) **Общая свёртка.** Для произвольных алгебр

  $$\alpha : F(C,D) \to C, \qquad \beta : G(C,D) \to D$$

  выведите уравнения, задающие совместную свёртку $h : T \to C$ и $k : U \to D$. Выразите обе функции через обычные катаморфизмы одноаргументных функторов из первого пункта. Выпишите все промежуточные алгебры и их носители. Докажите, что полученная пара удовлетворяет выведенным уравнениям и определяется ими единственным образом. В окончательных формулах используйте катаморфизмы, композицию и действие функторов на функциях.
- (3 балла) **Закон слияния.** Рассмотрите вторую пару алгебр с носителями $C', D'$ и функции $p : C \to C'$, $q : D \to D'$. Сформулируйте условия согласованности алгебр и выведите закон, связывающий свёртки с преобразованием результатов с помощью $p, q$. Докажите закон для общего случая.
- (3 балла) **Применение.** Выполните построение для

  $$F(X,Y) = 1 + A \times Y \times Y, \qquad G(X,Y) = B + X \times Y.$$

  Запишите исходные ADT и полученные представления на OCaml, Haskell или типизированном псевдокоде. Через общую конструкцию свёртки реализуйте две функции: для значения типа $T$ и для значения типа $U$. Каждая возвращает пару чисел – количество конструкторов типа $T$ и количество конструкторов типа $U$ во всём значении. Проверьте функции на собственном примере хотя бы с шестью вхождениями конструкторов типов $T, U$. Сопоставьте результат с прямым подсчётом.

Общая часть решения должна содержать явные типы, формулы и доказательства. При проверке конструкции рассмотрите также случаи с пустыми типами меток и с нулевыми показателями степеней.